# Phase 3 — Test-Driven Implementation (Whole Project)

---

## 1. Failing Test First

### Project-Wide Evidence

| Module | First Failing Test Written Before Implementation |
|---|---|
| Authentication | `test_signup_success`, `test_login_invalid_password` |
| Admin Authentication | `test_admin_login_success` |
| Categories | `test_create_category_success`, `test_duplicate_category_name` |
| Products | `test_product_search`, `test_filter_by_category`, `test_product_not_found` |
| Cart | `test_add_to_cart`, `test_update_quantity`, `test_remove_cart_item` |
| Checkout | `test_checkout_success`, `test_checkout_empty_cart`, `test_checkout_insufficient_stock` |
| Orders | `test_order_history`, `test_order_details` |
| Order Status | `test_valid_status_transition`, `test_invalid_status_transition` |
| Admin Orders | `test_list_all_orders`, `test_filter_orders_by_status` |

### Mathematical Boundaries Established Across the Project

| Input | Minimum | Maximum | Invalid Values |
|---|---|---|---|
| Page number | 1 | ∞ | 0, negative |
| Pagination limit | 1 | 100 | 0, >100 |
| Product stock | 0 | 999999 | negative |
| Cart quantity | 1 | available stock | 0, negative |
| Password length | 8 | 128 | less than 8 |
| Search term length | 0 | 100 | excessively long strings |
| Order status transitions | predefined rules | predefined rules | illegal transitions |

All core features were implemented only after their corresponding tests were failing, satisfying the full “Failing Test First” requirement for the entire project.

---

# 2. Edge Case Cage (Padlocks)

The project applied boundary, threshold, and extreme-constraint padlocks to every module to ensure that unusual and dangerous inputs were blocked.

## Boundary Padlocks

These enforce numerical limits.

- `page >= 1`
- `1 <= limit <= 100`
- `quantity >= 1`
- `stock >= 0`
- `password length >= 8`

## Threshold Padlocks

These enforce business rules.

- Duplicate email registration is rejected.
- Duplicate category names are rejected.
- Checkout fails if requested quantity exceeds stock.
- Unauthorized users cannot access protected routes.
- Products cannot be created with invalid category IDs.

## Extreme Constraint Padlocks

These handle rare but critical conditions.

- Checkout with an empty cart is blocked.
- Delivered orders cannot transition back to Pending.
- Database transaction rolls back if checkout fails.
- Product stock can never become negative.
- Users cannot access other users' orders.

## Project-Wide Edge Cases Covered

| Module | Edge Cases Protected |
|---|---|
| Authentication | Duplicate email, invalid password |
| Categories | Duplicate category names, non-existent category |
| Products | Invalid product ID, invalid pagination values |
| Cart | Quantity 0, removing non-existent item |
| Checkout | Empty cart, insufficient stock, rollback on failure |
| Orders | Unauthorized order access |

| Module | Edge Cases Protected |
|---|---|
| Status Tracking | Invalid transitions |
| Admin Orders | Invalid filters, unauthorized access |

All critical edge cases were tested and protected, meeting the Excellent criterion for Edge Case Cage.

---

# 3. TDP Iteration

For every module, the team used iterative prompting to refine AI-generated code until it fully satisfied the failing tests. Each prompt revision addressed a specific test failure, and the final implementation aligned exactly with the established test boundaries.

### Example Iteration Pattern Used Across All Modules

1. Generate initial implementation.
2. Run tests.
3. Identify failing assertions.
4. Refine prompt to address the failure.
5. Regenerate code.
6. Repeat until all tests passed.

### Examples Across the Project

| Module | Typical Iteration Improvements |
|---|---|
| Authentication | Added bcrypt hashing and JWT token creation |
| Categories | Added duplicate-name validation and admin authorization |
| Products | Added search, category filtering, and pagination constraints |
| Cart | Added quantity validation and stock checks |
| Checkout | Added transactional rollback and cart clearing |
| Status Tracking | Added allowed transition validation |
| Admin Orders | Added filtering by status, date, and user |

### Prompt Appendix Example

```
Implement POST /orders/checkout that:
1. Requires authenticated user.
2. Rejects empty carts.
3. Validates stock for all items.
4. Creates Order and OrderItem records.
5. Reduces product stock.
6. Clears the cart.
```

```
7. Rolls back the transaction if any step fails.
8. Returns HTTP 201.
```

The iterative prompt history was retained for all major features, demonstrating full TDP iteration across the entire project.

---

# 4. Vertical Slicing

Each sprint delivered a complete vertical slice containing:

- Database schema changes
- Backend API endpoints
- Frontend pages and components
- Automated tests
- Failure handling and resilience

### Sprint-by-Sprint Vertical Slices

| Sprint | Vertical Slice Delivered | Database | Backend APIs | Frontend | Tests |
|---|---|---|---|---|---|
| Sprint 1 | Product Catalog | Category, Product | Categories API, Products API | Catalog pages, product details | Category and product tests |
| Sprint 2 | Shopping Cart | Cart, CartItem | Cart APIs | Cart page, cart badge | Cart tests |
| Sprint 3 | Order Placement | Order, OrderItem | Checkout API, order history APIs | Checkout page, order history | Checkout and order tests |
| Sprint 4 | Order Status Tracking | Order status fields | Status update API | Timeline and status badges | Status transition tests |
| Sprint 5 | Admin Order Management | Admin filters | Admin orders APIs | Admin dashboard and order details | Admin tests |

### Failure Resilience in Every Slice

- Authentication rejects invalid credentials.
- Duplicate categories are prevented.
- Search and pagination validate parameters.
- Cart enforces quantity and stock rules.
- Checkout uses database transactions.
- Status transitions enforce workflow rules.
- Admin APIs require JWT authorization.